



M362 Unit 12
UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



Mobile systems

Unit **12**

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

ISBN 978 0 7492 1600 9

1.1

CONTENTS

1	Introduction	5
1.1	The aims of this unit	6
2	Physical mobility	7
2.1	Wireless communication	7
2.2	Mobility	9
2.3	Physical portability	12
2.4	Development of mobile systems	13
3	Logical mobility	14
3.1	The client-server paradigm	15
3.2	Remote evaluation	16
3.3	Code on demand	18
3.4	Mobile agent	20
3.5	Strong and weak mobility	22
4	Economic and social impact	24
4.1	Economic impact	24
4.2	News and commentary	24
4.3	Privacy	25
4.4	Developing countries	26
5	Distributed computing: the fallacies and the future	28
5.1	The fallacies of distributed computing	28
5.2	The future of distributed computing	31
6	Summary	34
	Glossary	36
	References	38
	Acknowledgements	39
	Index	40

M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

Unit 1 provided a brief overview of systems in which components are not only distributed (i.e. located in different places) but also mobile (i.e. the components' locations are constantly changing). *Unit 1* also mentioned some of the technologies used in such systems and outlined the costs and benefits of mobile systems.

This unit will extend the treatment of mobility presented in *Unit 1* and further discuss technological, but also economic and social, issues raised by mobile systems. Mobile computing is a rapidly developing area with many new technologies and services being constantly developed. In a single unit we can hence give just a flavour of this exciting area.

Unit 1 defined mobile systems as distributed systems in which some components may not have a fixed location. There are, however, two possible definitions for 'component' and 'location', and hence two kinds of mobility, called physical mobility and logical mobility. *Unit 1* introduced physical mobility and this unit will introduce logical mobility.

If components are hosts and locations are points in some physical space, then we are talking about **physical mobility**. For example, using a PDA with an internet connection in order to regularly check, from anywhere we are, the latest bids on the musical instrument auction site, is an example of the advantages of physical mobility. In this unit, the kinds of mobile host we are interested in are networked computing devices like laptops, mobile phones (also called cellular phones or simply cell phones), PDAs (personal digital assistants) with network access, and smartphones (the convergence of mobile phones and PDAs). Stand-alone, dedicated, portable computing devices, like MP3 players, are of marginal interest to us as they are not networked: their 'communication' amounts to transferring files when connected to a computer.

If instead of hardware components moving around the physical space we have software components moving across (possibly fixed) hosts, then we are talking about **logical mobility**. For example, many multimedia and interactive websites include code that is transparently downloaded and executed by the browser on your desktop machine. In this case, neither the server nor the desktop are mobile; the mobile system is the overall distributed application, which includes some software components that change location.

Although some of the issues to be covered, like location awareness and privacy, are typical of mobile systems, they are not exclusive to such systems. This unit will therefore occasionally sidetrack to non-mobile (but distributed) systems, in order to emphasise the wider scope of such issues. In particular, we are also interested in small wireless devices, like sensors, that perform few or no computations but enable the development of distributed systems that, while not necessarily mobile, share some characteristics of mobile systems.

1.1 The aims of this unit

This unit aims to cover some of:

- ▶ the essential properties of physical mobility, the technological challenges they pose and the solutions that are available (Section 2);
- ▶ the design paradigms used by logical mobility and how the JVM supports them (Section 3);
- ▶ the social and economic consequences of mobile systems (Section 4);
- ▶ the common misunderstandings about distributed computing and how they are relevant to mobile systems (Section 5).

This unit does *not* aim to provide any practical experience in developing mobile systems. In fact, its precise purpose is to step back from details and to look at mobility from a wider perspective.

This unit is shorter than the previous ones because it is meant to be complemented by other reading that you will carry out as part of the activities. The activities may ask you to write notes or summaries of what you read: these might prove useful when writing your assignment answers or when revising for the examination.

We strongly advise you to reread Subsections 5.2 and 6.4 of *Unit 1* before continuing with this unit.

2

Physical mobility

Having just read *Unit 1*, you will have noticed that it stated that wireless communication is not a requisite for mobile systems: e.g. a laptop might be disconnected from the fixed network at the office and then connected to a broadband cable at home. However, the full potential of physical mobility is harnessed only when there are no cables attached, i.e. with wireless communication. Hence, in this unit we restrict ourselves to the subset of mobile systems that are also wireless. In particular, we will define **mobile computing** as the use of a portable computer with a wireless connection to a network. This definition captures the three essential properties that differentiate mobile systems from the distributed systems we have seen so far: wireless communication, mobility and portability (Forman and Zahorjan, 1994). In this context, **portability** means the ability to move the computational device easily from one location to another.

SAQ 1

The term 'portability' has occurred previously in this course, but with a different meaning. Which one?

ANSWER.....

Portability also means the ability to easily adapt an application in order to make it run on various platforms. We encountered this term in *Unit 4* in the context of the portability of applications written to run on top of a virtual machine.

Normally, the different usages of 'portability' should be clear from the context, but to avoid any misunderstandings, we will use in this unit the terms **physical portability** and **application portability**.

In the remainder of this section we will see the challenges that the above three properties entail.

2.1 Wireless communication

Wireless communication is usually achieved using microwaves, radio waves or infra-red light, each suited to different purposes. For example, microwaves are used for long-range communication, such as that offered by WiMAX (see *Unit 1*, Table 3), while infra-red light is mainly used to establish direct communication between two mobile computers that are in close proximity, because the infra-red emitter and receiver have to be in direct 'line of sight' of each other.

Infra-red transmission is convenient to share information quickly (e.g. a calendar appointment) between two PDAs or two laptop computers, or to connect a laptop computer to a mobile phone. However, due to the physical constraints of infra-red communication, microwaves and radio waves are most commonly used. The wireless communication can be linked to the wired network infrastructure via stationary **transceivers**, which emit and receive radio signals to and from the mobile computers within the transceiver's **cell**, the physical area delimited by the range of the transceiver's radio signal. The transmission of radio waves is subject to interference from the physical environment (e.g. buildings), which may lead to blocking, noise or

echoes. Wireless communication is therefore subject to higher rates of error and disconnection than wired communication. The underlying communication protocols must therefore incorporate sophisticated error control, timeout and retransmission mechanisms to cope with those problems.

Moreover, the communication bandwidth (often measured by the number of bits that can be transmitted per second) is normally much lower for wireless communication than for wired communication, not only because of intrinsic physical limitations but also because the bandwidth is shared by all users within the same cell. The challenge is therefore to provide an acceptable level of service under such constraints. Techniques to address this problem may be at the infrastructure level (e.g. by using data-compression algorithms before transmission) or at the application level (e.g. an email client may download new email in the background before notifying the user of new incoming email).

A further problem of wireless communication is the increased security risk, already mentioned in *Units 1* and *10*. To eavesdrop on a message transmitted over a wired network, the intruder needs to gain access to a computer relaying that message. With wireless communication, the intruder can easily find any transceivers within range and then listen to their communication. Moreover, users might be lured to connect to 'fake' networks, which actually log the users' activity. Using the techniques discussed in *Unit 10*, like requiring passwords to establish a connection and encrypting all communication with stream ciphers, goes a long way to minimising the security risk.

Several technologies for wireless communication have been designed to cope with bandwidth and security issues. Each technology addresses different concerns and it is unlikely that there will ever be a single universal wireless communication technology fit for all purposes, just as no one programming language suits all requirements. Table 3 in *Unit 1* introduced some of the wireless communication technologies, covering various bandwidths and ranges: Bluetooth is used for short-range **personal area networks** (e.g. to connect mobile phones to laptops), WiFi is used for local area networks and WiMAX is used for wide area networks (e.g. covering part of a city). But wireless communication also makes other kinds of network possible.

Ad-hoc networks allow various computing devices to establish a direct communication among them, i.e. without using an external network provider, usually in a peer-to-peer fashion, without a dedicated router. Consider, for example, an impromptu business meeting during breakfast in a hotel. The meeting participants might wish to establish an ad-hoc network directly between their laptops for the duration of the meeting to exchange documents and work collaboratively. They can establish this network directly, even if the hotel provides no WiFi network, by using the laptops' built-in WiFi cards to transmit and receive information from each other. The ad-hoc network can also be established in spite of any existing hotel network, e.g. to avoid potential eavesdroppers on that network. Although the nodes in an ad-hoc network may be stationary while connected to the network, ad-hoc networks often have mobile devices in mind, because they make it easy for nodes to enter or leave the network. For our example, the network grows as people arrive for breakfast and progressively shrinks as meeting participants leave. One of the challenges in ad-hoc networks is to develop efficient and robust message-routing protocols that cope with the changing connectivity of the network. This means that routing paths are computed dynamically and cannot rely on centralised routers: normally, each node in an ad-hoc network is a potential router.

Another kind of network is a **wireless sensor network**. Compared to the wireless networks we have seen so far, which connect complex computational devices like laptops and smartphones, sensor networks connect small devices that provide

information about the environment to control applications and are used in factories and home automation. Although sensors may be attached to objects that move around, often sensors are fixed to certain locations. A wireless sensor network therefore often does not constitute a mobile system.

Exercise 1

Even if the system is not mobile, wireless sensor networks are usually ad-hoc. Why?

Discussion.....

Being ad-hoc allows the network to automatically accommodate the addition and removal of sensors. Moreover, ad-hoc networks are more robust to node failure, which in this case means that routing can dynamically bypass any sensors that are not working.

Although wireless sensor networks may use existing protocols, like WiFi and Bluetooth, special purpose protocols are being developed. ZigBee is such a protocol, defined on top of the IEEE 802.15.4 standard for wireless communication. ZigBee aims to fill a niche for low data rate applications: it has a lower bandwidth, but also consumes less power and requires less software than Bluetooth. ZigBee is therefore adequate for wireless networks connecting small remote sensors that have to continuously operate on a battery for a long time. ZigBee also allows thousands of devices to build up an ad-hoc network. By contrast, Bluetooth only allows networks of up to eight active devices, with one master device communicating individually with each of the seven slave devices, although devices may switch roles.



ZigBee Alliance
tutorials.

By their very purpose, wireless sensor networks are inconspicuous and embody the ideal of ubiquitous computing, as proposed by Weiser (1991):

Most of the computers [...] will be invisible in fact as well as in metaphor. Already computers in light switches, thermostats, stereos and ovens help to activate the world. These machines and more will be interconnected in a ubiquitous network.

Although we have taken wireless communication to be a core characteristic of mobile computing, it should in fact be seen more generally as an essential component of networked ubiquitous computing.

2.2 Mobility

Node identity

The ability for a computational device to change location while communicating also introduces problems of its own. If the device is moving across cell boundaries within the same network, the ongoing communication has to be 'handed over' from the current transmission channel in the current transceiver to an available channel in the new cell. Since the computer is staying within the same network, this movement involves only the physical and data link layers of the communication protocol (*Unit 5*, Subsection 3.2).

More problematic is the case when the mobile computer is moving from one network to another one (e.g. from a corporate intranet to the network of an internet service provider) and therefore changes its IP address. The main challenge here is to properly route any ongoing communication to the current address in an efficient manner,

i.e. avoiding disruption or too much overhead on the network. This is a radical shift from distributed systems with a fixed network topology, which was the assumption for the development of the original Internet Protocol (IP) that underlies communication between nodes. A new protocol had therefore to be devised to cope with mobile nodes, and **Mobile IP** was defined by the Internet Engineering Task Force, the international organisation that proposes changes to the architecture and protocols of the internet. In essence, Mobile IP works as follows (Ghosh, 2000).

Unlike fixed nodes (i.e. computers), which have a single fixed address, each mobile node has two addresses: its permanent address in the home network and a 'care-of' address in the current network that the node is visiting. Information about the mobile node is kept by two entities: a home agent in the home network and a foreign agent in the current network. When a node moves into a new network, it has to find out that network's foreign agent and obtain from it a care-of address. Usually, the care-of address is the address of the foreign agent. The mobile node then transmits this address to the home agent, which updates the information it holds. When a computer wants to communicate with the mobile node, it sends the packets to its permanent address, as usual. The home agent intercepts those packets and re-routes them to the node's care-of address. The foreign agent receives the message and forwards it to the mobile node. Since the packets contain the IP address of the sender, the mobile node can then communicate back to the sender. It is possible for the reply to go back to the home agent and then to the sender – in which case the sender remains unaware of the mobile node's care-of address and keeps using its permanent address – or to go directly from the foreign agent to the sender – in which case the sender uses directly the mobile node's care-of address for further communication. The first option makes communication less efficient due to the added indirection, but it makes it easier for the mobile node to move to a new foreign agent, as it requires only the home agent to be notified. In the second option, all senders communicating directly with the current foreign agent have to be notified of the new care-of address.

Exercise 2

Recalling what you learned in *Unit 10*, can you think of a security problem related to the mobile node's transmission of its new care-of address to the home agent?

Discussion.....

One security problem is authentication, in order to assure the home agent that the new address is indeed coming from the mobile node. Otherwise, a hostile node could pose as the mobile node and transmit its own address to the home agent and therefore start getting all communication intended for the mobile node.

Location awareness

The ability to tailor a system's behaviour to the user's location is not restricted to mobile systems. For example, when browsing the internet from a fixed location (e.g. a desktop machine at home), you may have come across websites that, based on your IP address or using other techniques, adapt to your country by changing such items as the available products or the licence agreement wording. Especially in the case of international businesses, instead of having one single location-aware website, they may have multiple 'local' websites, each one able to redirect you to the one appropriate for your country.

Another example of a location-aware system comes from Portugal: Via Verde [green lane] is the first ever system to be used to pay *all* the motorway and bridge tolls of a country. A user of this system installs a small **radio-frequency identification (RFID)** device on their car's windscreen, and they have to pass through dedicated toll lanes. Each device has a unique ID, which is associated in the Via Verde database with the car's licence plate and the user's bank account. Whenever the car enters a motorway through a special toll lane, an antenna writes into the device the entry point (e.g. junction 6 of motorway 4). When the car is exiting the motorway, an antenna reads that information and the device's ID. This makes it possible to calculate the due toll – which depends on where the user entered and exited the countrywide network of motorways – and debit it from the user's account. The system's implementation is of course not simple, because it has to detect fraud, e.g. a device bought for a passenger car is being used by a lorry that should pay a higher toll. The convenience of not having to stop at toll booths and not having to carry cash made the system very popular, which in turn made further applications of Via Verde possible: it is currently also used by several parking places and petrol stations.

All transport-related systems are by their nature location aware. As a further example we mention the congestion charge enforcement system used in London. Users travelling by car into central London during working hours must register on the congestion charge system their car's licence plate. The system has cameras throughout central London, supported by licence plate recognition software and human employees, to monitor all vehicles. The system checks them against the database of registered vehicles and issues a fine if there is no match.

RFID devices can store a small amount of data and return it on receiving a radio signal. They are widely used, e.g. within passports, car keys and books.



Radio-frequency identification.

The Active Badge project

One of the earliest location-aware systems was the research project Active Badge in the early 1990s at Olivetti Research in Cambridge, UK. A person carried a badge with a button around the building. The badge emitted an infra-red signal that was picked up by sensors in the building. Originally, the system was intended to enable calls to be automatically routed to a phone in the room where the callee was located, and the button was intended just to test the system: pressing the button would flash indicators in the sensors to show that they were working (Want and Schilit, 2001). However, soon the researchers found that the Active Badge could be put to other uses, in spite of its simplicity. For example, given that the system knew on which badge in which room the button was pressed, the button could be used to issue simple commands that depended on the user *and* the location. In one system developed by the researchers, pressing the button would show on the nearest workstation terminal a personalised list of applications that the user was authorised to run.

Exercise 3

Discuss whether Via Verde, the London congestion charge system and active badges fit our definitions of mobile systems and mobile computing.

Discussion.....

All of these systems are mobile in the general sense because essential components (i.e. the vehicles and badges) do not have a fixed location. In particular, the systems detect when those components enter a controlled area, be it a motorway, central London or an office. However, no computation takes place in the mobile components: even in the case of Via Verde, the RFID device just receives and transmits a small amount of data. And in the case of the congestion charge system, there is not even wireless communication: it is definitely not a mobile computing system. Neither are

active badges, due to the lack of any data processing on the badge. However, Via Verde may be considered on the borderline, depending on whether we view RFID devices capable of data input and output as computational devices.

Although non-mobile systems may be location aware, the use of location-dependent information becomes more relevant for computational devices that change their location. For example, the list of available printers and the local time zone can be configured statically for desktop computers, but mobile computers should be able to find such configuration information dynamically: this applies, for example, to laptops when first linked (with or without wires) to a particular local network. Moreover, the mobile wireless case raises some additional possibilities for location-dependent information. Mobile devices can help guide users in unfamiliar places. For example, users might wish to know where the nearest cash machine or Italian restaurant is. Such **geotagged information** requires the ability to find out the location of the user.

The cell where the mobile node is can serve as an approximate location. For more precision, a **Global Positioning System (GPS)** receiver is necessary. GPS makes use of several satellites that continuously send microwave signals to Earth, indicating the precise time and the satellite's location. A GPS receiver measures the delay in receiving the signal from three satellites, and from that computes its precise current location in terms of latitude and longitude. GPS was originally developed for the US military but receivers are currently widely available in mobile phones, PDAs and car navigation systems.

The disclosure of a user's location is useful or even mandatory for several applications. For example, the Enhanced 911 requirements, enforced by the Federal Communication Commission in the US, state that wireless phone companies have automatically to provide to emergency dispatch services the phone number and an approximate location (within 50–300 metres depending on the technology) of those calling the 911 emergency phone number.

Precise time information was covered in *Unit 9*, Subsection 6.1.

2.3 Physical portability

Ideally, mobile devices should have the same characteristics as a wristwatch: small, light, robust and with a long battery life. Unfortunately, this is easier said than done for mobile devices, as these requirements may conflict, thus necessitating various trade-offs depending on the particular application.

For example, a small device means a small screen, which leads to various usability and human–computer interaction issues requiring different ways of presenting and inputting information: instead of a mouse and keyboard, PDAs and tablet PCs use a stylus, a touch screen and handwriting-recognition software. The heat dissipated by the CPU also becomes a bigger problem for small devices. Although great advances have been made in the past years, batteries still remain a problem. Not only are batteries the largest component by weight for small devices, they also need frequent recharging. Various techniques can be used to diminish power consumption, like reducing the CPU's clock frequency and voltage. The former leads to a reduced computational speed, while the latter requires new chips to be developed. Other techniques involve clever usage of available resources. For example, if there is no user interaction for some time, the screen can be switched off. Also, radio transmission requires much more power than reception, and by using push instead of pull technologies (*Unit 9*, Section 5) power may be saved.

The different use of memories in desktop and handheld computers illustrates how portability impacts on many issues. Desktop machines use fast RAM memory chips for volatile memory, i.e. memory that does not keep data once the power is switched off, and large hard disks for non-volatile memory. Hard disks have moving parts operating under high precision and may fail due to humidity, power loss or sudden movement. Moreover, hard disks consume much more power than memory chips. Mobile devices have to be rugged – they are carried around in pockets and are used outdoors – and are switched off and on frequently. Therefore they tend to use flash memories, which are solid state (i.e. without moving parts) and non-volatile. This not only saves power, it also eliminates long reboot delays. The disadvantage is (at the time of writing) that flash memories are slower than volatile memory chips and much more expensive (per gigabyte) than hard disks. Thus, handheld devices usually have a much smaller memory capacity than laptops, but continuous technological developments may change this state of affairs.

Activity 12.1
Mobile computing.

2.4 Development of mobile systems

Let us now consider the problem of developing software for physically mobile systems – we address logical mobility later. Two of the main issues are application portability and **context awareness**. Portability is important because, even considering the same brand of devices, there is a wide range of models with differing features (e.g. with or without a camera), and we would like the same application to work on all models that include the features required by the application. On the other hand, applications will only take advantage of the device's mobility if they are aware of the current context of operation, which includes not only location but also any other factor that might impact the application's behaviour. For example, if battery power is low, the application might reduce the need for wireless data transmission, or if the mobile phone is in 'silent mode', the application might hold alarms until the phone returns to normal mode.

However, as Huebscher et al. (2006) report for the case of Symbian phones, the two issues are somewhat conflicting. Applications can be context aware only if they use the operating system calls that allow querying the phone's current battery level, operation mode, etc. However, making these low-level calls makes the application non-portable, as the operating system version varies from model to model. On the other hand, using a platform like Java ME (*Unit 4*, Subsection 2.2) makes the application portable, but prevents access to contextual information that is not provided by the middleware's API. All this is further compounded by the fact that Java ME is flexible and allows phone manufacturers to implement different Java ME configurations, thereby decreasing the application's portability.

Symbian is a multithreading, pre-emptive multitasking operating system for handheld devices, used by Nokia, Ericsson and other mobile phone manufacturers.

Further software development problems include the restricted availability of sophisticated integrated development environments (IDEs) and simulators that allow programmers to develop applications on their desktops and test them confidently before deploying them on mobile devices.

However, as mobile applications become more context aware, we can expect the above-mentioned shortcomings to be addressed, making the life of mobile application developers easier.

Activity 12.2
Software development for mobile systems.

3

Logical mobility

In the previous section we have seen some of the issues that have to be addressed in order to develop computing devices that are physically mobile. In this section we turn to **mobile code**, i.e. the ability for code to be transferred to and then executed on a different device.

SAQ 2

Mobile code has already appeared in previous units. Give an example.

ANSWER.....

A typical example of mobile code is an applet (*Unit 9*, Subsection 3.1), which is transferred from the server to the client's machine and executed there.

In *Unit 5* we saw different paradigms for distributed code to communicate; now we will look at paradigms where the code itself moves, instead of just exchanging data. We will follow, with slight simplifications, the concepts and terminology introduced by Carzaniga et al. (2007). We will also adapt their informal example of two friends, let us call them Anton and Brendan, involved in the baking of a chocolate cake, in order to illustrate the various paradigms.

Computation involves an *execution unit* (e.g. a process or a thread) executing *code* that uses some *resources* (notably data). Only when the execution unit, the code and the resources are at the same *site*, e.g. host machine, can the computation occur. In the case of baking a cake, the recipe is the code, the ingredients are the data, the oven and other utensils are the remaining resources, the person baking the cake is the execution unit, and the kitchen is the site.

Exercise 4

The above statement that code and resources must be located at the same site for computation to occur may strike you as strange, especially in a course about distributed computing. We have only to think of enterprise systems comprising an application server and a database on different hosts: the computation is done on the server and the data is held in the database. Can you explain in more detail what is meant by computation requiring the co-location of execution unit, code and resources?

Discussion.....

A distributed system consists of multiple execution units (processes and threads) executing concurrently in different hosts. However, each process or thread can only execute local code on local data. If the data is not available, usually the process sends a request to another execution unit and suspends while the other execution unit computes the necessary data. In the example mentioned, the application in the server machine does not manipulate directly the database tables in the other host. Instead, the application sends queries to a database manager, which executes them locally on the database and returns the results. The application then continues the computation with a local representation of the data, e.g. entities (*Unit 7*, Subsection 4.3).

The distinction between sites, code, resources and execution units makes the difference between non-mobile, physically mobile and logically mobile systems clear. In non-mobile systems, *only* resources may move; physical mobility is about the movement of sites; and logical mobility is about the movement of code from one site to another. Let us first see how to use these concepts in a familiar paradigm that does *not* involve mobility.

3.1 The client–server paradigm

In this paradigm, the client makes a request to the server, which has all the necessary resources and the code on the same site (Figure 1). The server will then return the results of the computation. Of course, the client host also contains code and resources used by the client, but we are just focusing on the computation done on the server machine and hence omit any other code and resources not relevant to that computation. However, we could argue that sometimes a part of the data needed by the server is passed as parameters of the request. We will gloss over such fine details and concentrate just on the major characteristics of the paradigm, which are, in this case:

- 1 the code and most of the resources are on the server side;
- 2 the only thing that moves around is some data representing the request and the reply.

The basic remote call functionality provided by RPC (*Unit 5*, Subsection 7.1) and RMI (*Unit 7*, Section 2) is an example of this paradigm. For example, recall the first version of the computation engine presented in *Unit 7*, Subsection 2.1: the client called a remote method on the server that computed the factorial for a given small integer. This is a typical instance of the client–server paradigm: a small integer is passed as part of the request, and its factorial is returned as the reply.

In the cake-baking scenario, the client–server paradigm corresponds to Anton calling Brendan and asking him to bake the cake. Brendan has the recipe, the ingredients and all required utensils in his kitchen and hence can bake the cake and then give it to Anton.

The cartoons in the figures in this section are based upon those used in the presentation of Carzaniga et al. (2007), with the permission of the authors.

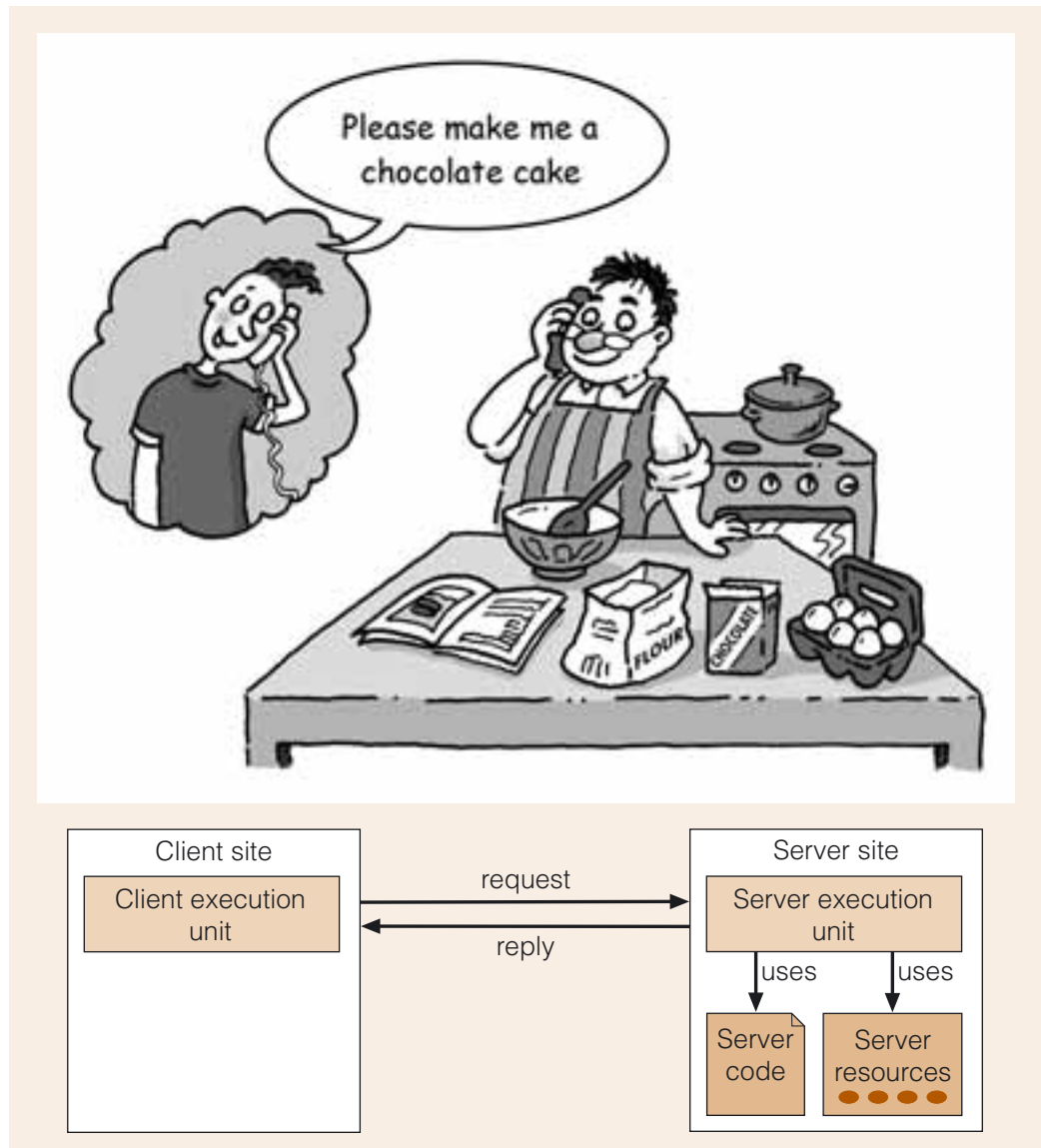


Figure 1 Client-server architecture

We next move to mobile code paradigms. Although they blur the distinction between what is a client and what is a server, due to the different usage of execution units, code and resources, we will keep using those two terms, with the sense that the client is the process starting the interaction.

3.2 Remote evaluation

Let us suppose that Anton has seen a chocolate cake recipe in a magazine and it sounds absolutely scrumptious. Lacking the talent or the ingredients to follow the recipe, he calls his friend Brendan, who is keen on baking, and tells him the recipe. Brendan can now bake the cake in his kitchen and then present the cake to Anton. In computational terms, code (the recipe) moves from the client site to the server site, where it is executed by a local execution unit (Brendan), using local resources (Brendan's oven and ingredients). This is called **remote evaluation** of the code. An example, arising from the discussion of Exercise 4, is the remote evaluation of an SQL query by a database management system, which then returns the result of the query.

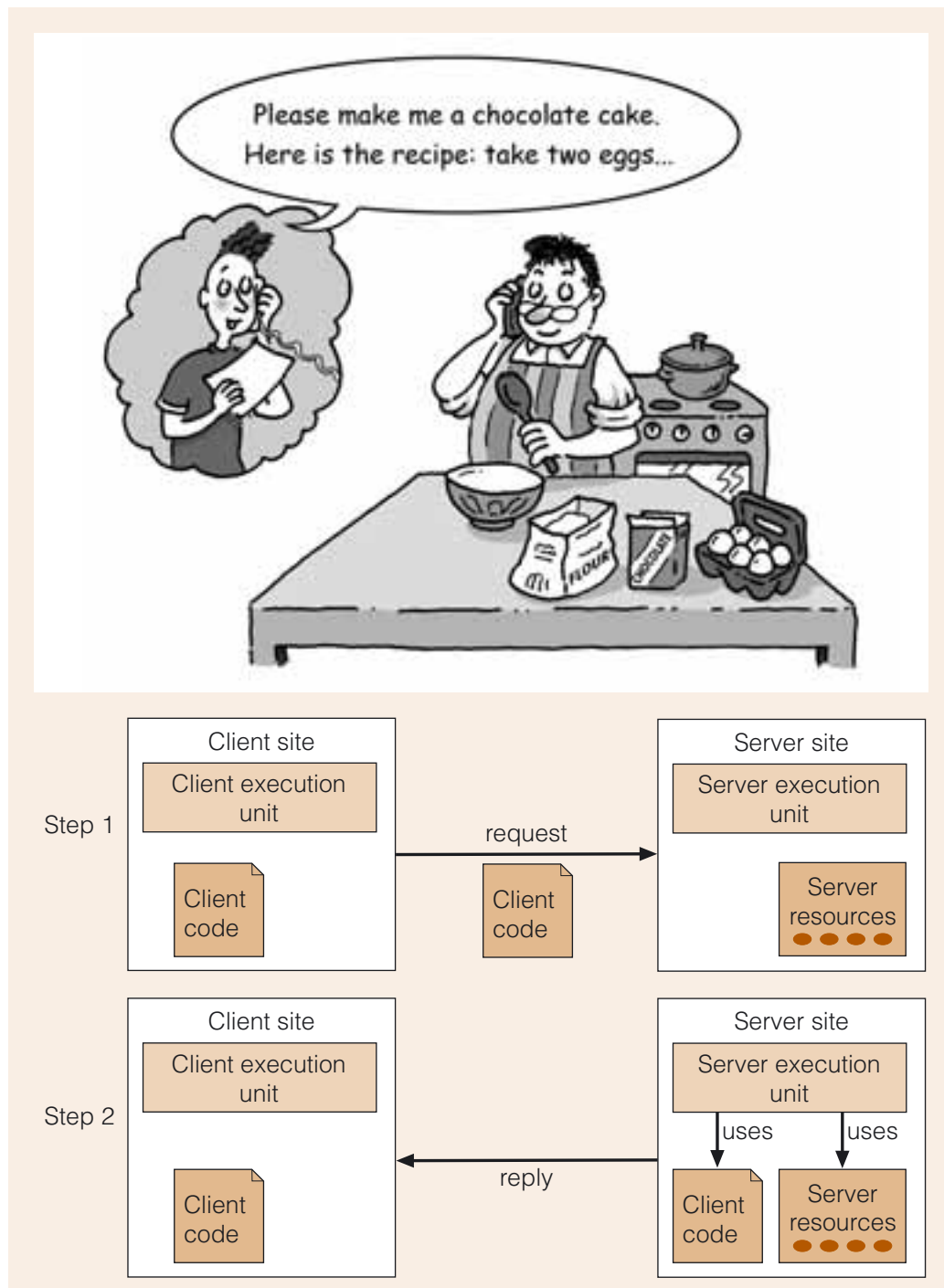


Figure 2 Remote evaluation

Although RMI is often used just as an object-oriented counterpart to RPC, it can lead to a richer interaction between client and server.

SAQ 3

Briefly skim again the computation engine example presented in *Unit 7*, Subsection 2.4: it allowed tasks, like the computation of the factorial, to be submitted to the server. Instead of implementing all possible tasks beforehand, the server just defined an interface `Task` with a method `compute`, delegating the implementation to the client, according to its needs. The computation engine consists of a single generic remote method `execute` that takes a `Task` object as parameter and calls the

`compute` method on the object. In the example, the client implements the interface through the class `Factorial`, instantiates it for a particular value, and then passes the `Factorial` object as an argument of the remote call to `execute`. Explain why the computation engine and its clients are an instance of the remote evaluation paradigm.

ANSWER.....

This is a typical remote evaluation scenario, because the task's methods are defined by the client but executed by the computation engine on the server. In the concrete example, when the computation engine receives a `Factorial` object for the first time, it loads the class (i.e. the methods' code) into the server's JVM, using the `codebase` property to locate the `Factorial` class on the client. Now the computation engine is able to run locally the `compute` method of the class `Factorial` and return the result to the client.

3.3 Code on demand

Consider now the symmetric scenario: Anton has a well-stocked and well-equipped kitchen, but he lacks a good chocolate cake recipe. Luckily, he knows that a wonderful recipe has been passed down for generations in Brendan's family and therefore calls him up. In computational terms, comparing Figures 2 and 3 we see that whereas in remote evaluation the client was requesting a *result*, sending the necessary code along, the client is now requesting some *code*, which will be executed on the client's site. This is called **code on demand**. Applets are an example of this paradigm: the applet code is downloaded only when the user opens a web page that contains the corresponding `<APPLET>` tag (*Unit 9*, Subsection 3.2).

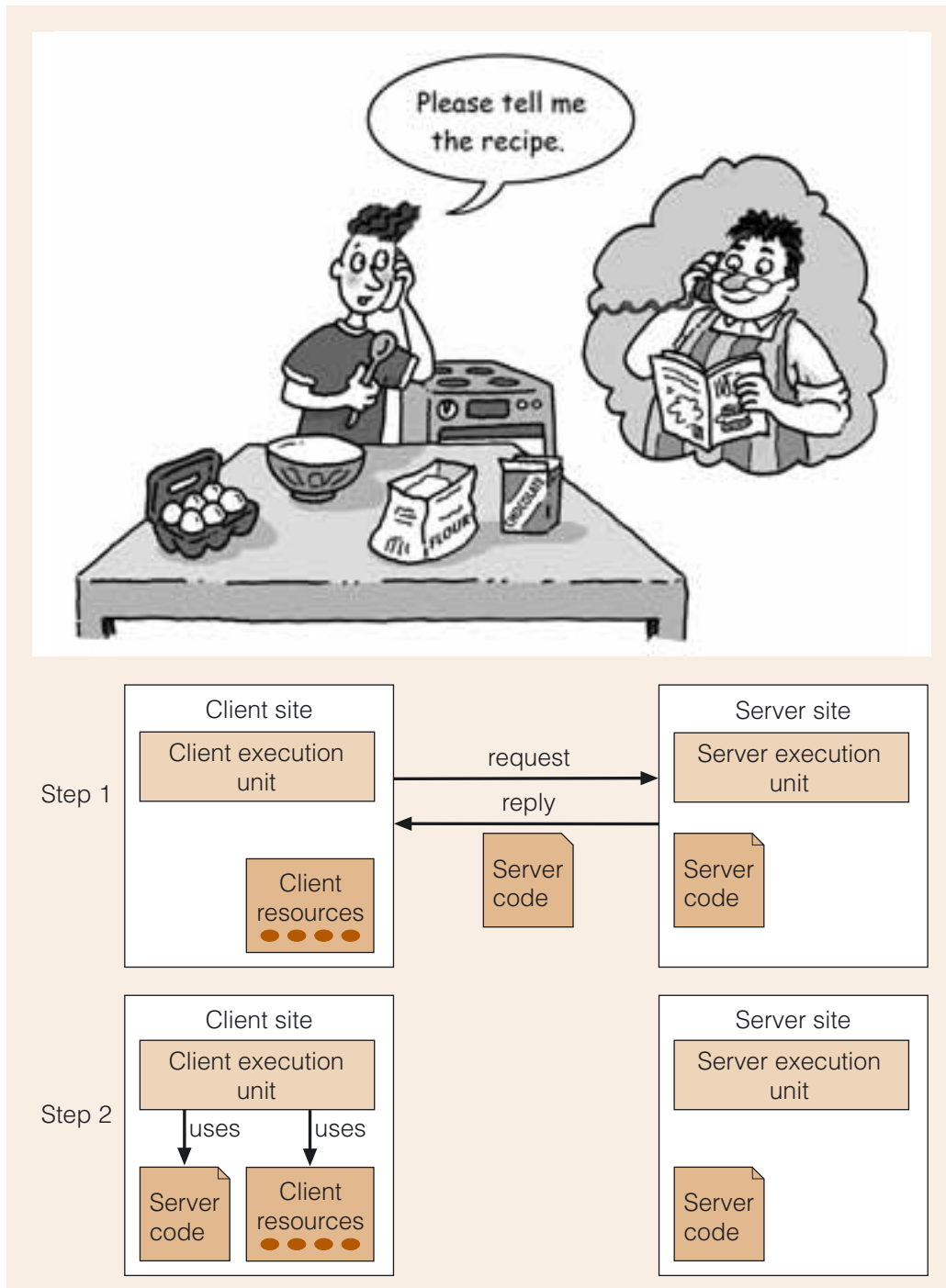


Figure 3 Code on demand

Java directly supports the code on demand paradigm through its class-loading facility (*Unit 4*, Subsection 2.2), which enables a JVM to load code (from the JVM's classpath or from external sources) into its memory only when needed.

Exercise 5

Does the classification of class loading as code on demand contradict the answer to SAQ 3?

Discussion.....

No, because the classification of an interaction depends on the scope of the interaction and, more importantly and as the figures above show, which side is starting it. In the

case of the computation engine example, the overall interaction between the client and the server is a remote evaluation, but it includes a code on demand interaction. In fact, once the server gets the first request, it has to get the `Factorial` code from the client: server and client switch roles temporarily, the server initiating the dynamic class loading and the client providing the code. Put in other words, at a conceptual level the computation engine example is an instance of remote evaluation, but at the implementation level it uses code on demand to 'pull' the `Factorial` class from the client to the server.

3.4 Mobile agent

In a third scenario, Anton has the recipe and the ingredients but his oven is not working properly. He therefore asks Brendan if he can come over to his place and use his kitchen to bake the cake. In computational terms, the execution unit (e.g. a whole process) and the code it is executing must be somehow suspended and transferred from the client site to the server site, where execution resumes, making use of further resources available on the server site. In this case we have a **mobile agent**, not just mobile code.

Figure 4 shows the execution unit being transferred with code and resources from one site to another, where further resources are available. Compared to previous figures, there is only one execution unit but there are two sets of resources – in the baking scenario, one represented by the ingredients and the other by the oven. Note that in the previous figures it was unnecessary to distinguish the ingredients from the oven, because they were always provided together by either Anton or Brendan, but in this case Anton provides the ingredients and Brendan the oven. In the scenario Brendan plays no role in the baking of the cake – diagrammatically this is reflected by the server execution unit being omitted from the third step of Figure 4.

The terms 'agent' and 'mobile agent' are also used in other contexts, e.g. artificial intelligence, with slightly different meanings.

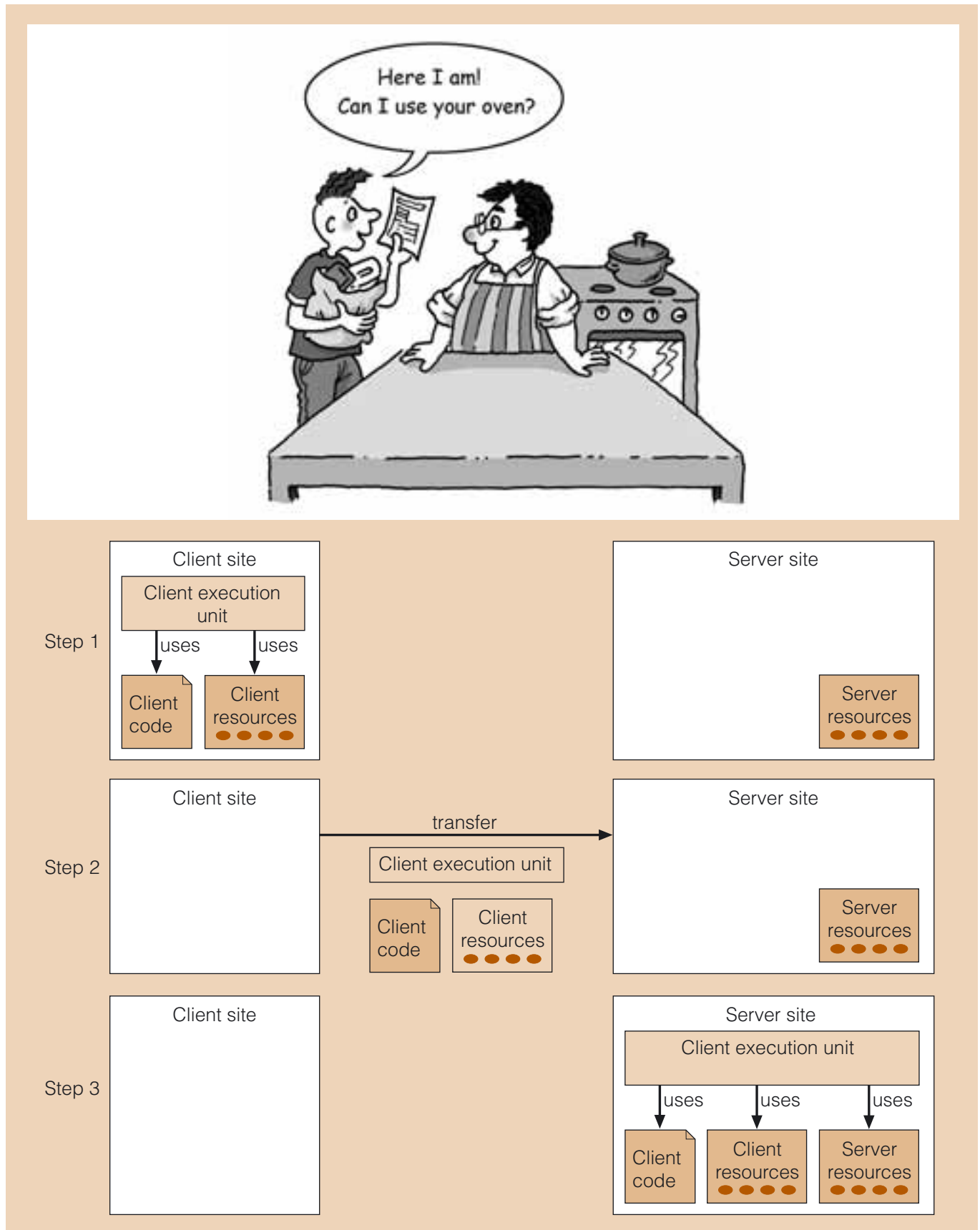


Figure 4 A mobile agent

As an example of a mobile agent system, we will take Nomad (Sandholm and Huai, 2000) because it is related to our music store scenario. Nomad aims to support more complex auctions than those offered by sites like eBay. Users can quickly configure predefined agent templates or program their own in Java. A user may install one, or more, agents at an auction site (e.g. one for each item the user is interested in), where they can bid on behalf of the user. Having the agents located in the auction host machines instead of the user machines reduces the need for network communication, making the system more efficient. Agents can also move from one auction site to another, e.g. to check prices for similar items, and can send notifications to a user about any relevant events (e.g. a new bid). Although much of the functionality of Nomad could also be achieved in other ways, having autonomous agents saves users from constantly logging into the auction site to keep track of current bids and possibly introduce a new, higher bid. Nomad supports combinatorial auctions, in which users can bid on different combinations of items. The number of combinations can grow exponentially, making a manual approach impractical. Nomad also allows a user to bid on similar items in multiple auction sites (sensibly ensuring that the user will win, at best, only one of the items – hence avoiding unwanted multiple purchases). Furthermore, an agent may be programmed to analyse the bidding behaviour of others and adjust its own bidding behaviour accordingly.

As Nomad illustrates, mobile agents can help build sophisticated systems that go well beyond the state-of-practice, but the vast majority have been research systems without commercial implementation. Two of the main reasons for the lack of wider adoption of mobile agents have been performance – do agents scale to millions of users? – and security – how can we avoid harmful agents? We might argue that security could be handled in the same way as for code on demand and remote evaluation. If we restrict the operations that applets can perform, why not do the same with agents? The reason is that, contrary to code which is a passive entity fetched and executed by some execution unit and then discarded, an agent is an active entity that has state and that moves of its own accord through various sites, possibly reading and writing data in each site. Mobile agents hence need a much tighter interaction with the resources of the site they are executing on than other mobile code paradigms. If the execution of mobile agents is constrained in the same way as code on demand or remote evaluation, then there is no point in having mobile agents: the same effect can be achieved by just moving code instead of an execution unit as well. In fact, we might argue that the most successful mobile agent systems have been computer viruses, because they are able to run and move freely across hosts by exploiting security flaws.

3.5 Strong and weak mobility

Of the three mobile code paradigms, the mobile agent paradigm is the most challenging, as it involves:

- 1 suspending an execution unit at its current site and capturing its current state (the values in the registers and program counter etc.);
- 2 transmitting the execution unit and the code to the new site;
- 3 resuming execution at exactly the same point in the new site.

The ability to perform these three steps is called **strong mobility**, whereas **weak mobility** is just the ability to move code (not a whole execution unit) between sites. Some operating systems might provide some form of strong mobility, for example to allow process migration among hosts in order to balance the load among machines. However, this is quite different from the aims of mobile code paradigms. The differences between process migration and mobile code are as follows.

- ▶ Whereas process migration is usually automatic and transparent to applications, i.e. under the control of the operating system, the mobility of code is controlled by applications.
- ▶ Mobile code works across the internet, while process migration is usually confined to a small local area network of machines.
- ▶ Process migration is based on the assumption that the process is not location aware and thus behaves in exactly the same way irrespective of which machine it is running on, while mobile code can exploit the current location to adapt its behaviour.

Implementation of a mobile agent system is greatly facilitated if the underlying platform or middleware supports strong mobility. If it only supports weak mobility, the mobile agent developer will have to somehow 'simulate' the above steps using the available facilities.

Exercise 6

From what you learnt in *Unit 4*, do you think that the JVM implements strong mobility?

Discussion.....

We have seen in *Unit 4* that the JVM has common memory areas for storing all the code and all the objects (the heap), and a per-thread area to store the program counter, registers and activation stack. However, the only mechanisms to transfer information across JVMs are object serialisation and class loading. Threads (execution units) can hence not be transferred from one JVM to another, which means that only weak mobility is supported.

Some mobile code paradigms have become more popular than others and Carzaniga et al. (2007) discuss various reasons for that. For example, code on demand is much more widely used than mobile agents, in part due to the scalability and security issues of mobile agents discussed above. A further reason is that code on demand requires only weak mobility, while mobile agents require strong mobility, thus making the former easier to implement. Moreover, an application that uses code on demand is easier to design and test than a mobile agent, which is a more complex and less predictable entity that has to decide how best to interact with the site it is running on, where to move next, etc. Finally, code on demand is enough for many of the current distributed applications and therefore the complexity overhead required for mobile agents is not warranted.

Activity 12.3

Mobile code paradigms.

4

Economic and social impact

Besides technological advances, mobile systems are having wide-ranging economic and social effects. We have already alluded to the security problems of wireless communication in Subsection 2.1. Such technical problems can have substantial social and economic impact, due to identity theft and revenue lost to fraud. In this section we discuss further social and economic problems, but also benefits, of mobile systems.

4.1 Economic impact

PDAs and small laptops allow people to work anywhere at any time, e.g. while commuting on the train. However, the potential economic advantages this entails, e.g. through increased productivity, may also be associated with potential social disadvantages: for example, by making it easier to take work home, laptops may lead to a decrease in work-life balance.

Nevertheless, mobile devices have obvious advantages for businesses that require their employees to visit clients. For example, it is not unusual, at least in the UK, for postal delivery workers to use a PDA to record the time of delivery and the signature of the recipient, when such proof of delivery is required, thereby eliminating the need for paperwork. Even if the PDA has no internet access, it is a simple matter at the end of the day to transfer the records on the PDA to the information system at the postal delivery centre. As another example, engineers sometimes bring laptops when doing a house call to repair some appliance. The computer might not only help with the diagnostics, but also allows the recording of the time spent, the repairs carried out, the parts that were used, etc. All this information can then help the company detect recurrent problems in particular models of their products, manage better the stocks of the most frequently needed parts, target staff development towards specific problems to reduce repair time, etc. All this could also be done with paper forms, but the efficiency gains of using a lightweight mobile device that can be carried in addition to all the engineer's tools are obvious.

4.2 News and commentary

The social impact of mobile devices has also been significant, and we are not just referring to the changes in communication habits that mobile phones and text messaging introduced.

The possibility of attaching a digital camera to a laptop with an internet connection, or the availability of smartphones with a camera and voice recording, has radically changed the way information is disseminated. Professional journalists can quickly write and edit reports that include text, image and sound on their laptops and send them to their headquarters. News travels faster than before: the websites of news agencies are kept up to date almost to the minute.

But even more importantly, technology has made it possible for information to be generated by everyone, not just by the relatively few working in the traditional media. For example, it is not unusual for TV news to use videos captured by ordinary people on their smartphones as they witness an event, before any news crew has a chance to arrive on location. The combination of mobile devices with websites that allow

anyone to post their photos, videos and text and to comment on other people's contributions makes it possible for information to be shared in an unprecedented amount and rate.

Although mobile devices are not a requisite for this general move from provider-generated to user-generated content, an important part of what makes the so-called Web 2.0, they certainly can contribute to it, because mobile devices allow content to be generated anywhere at any time, capturing sound and image at the point of inspiration.

Furthermore, Jaokar and Fish (2006) speculate that Web 2.0 together with GPS-enabled devices might be the ingredients needed to increase the availability of location-aware services, which so far have been restricted by the costs supported by the operators in order to mount the infrastructure and to populate the database with the geotagged information.

4.3 Privacy

Mobile systems also exacerbate concerns about privacy, and not only because of the security problems mentioned in Subsection 2.1.

Although location-related privacy issues have existed for a long time, e.g. historical data about credit card transactions can be used to trace the user's past movements, mobile devices make privacy concerns more relevant due to the ability to trace the location in real time and with greater precision. For example, there are services that allow subscribers to track the location of specific mobile phones, e.g. of employees or family members.

In contrast to such query-based services, others are disruptive: a London council is implementing a service that sends a text message to Bluetooth-enabled devices when they pass by disreputable places in Soho (BBC, 2007). The council's aim is to warn people, especially tourists, of potential dangers. Services such as this one raise a number of wider social issues, e.g. location-based spamming.

As a final example, wireless sensor networks like those connected via ZigBee and objects inconspicuously tagged with RFID devices also pose privacy issues, as they allow our movements to be monitored without us being aware.

Price et al. (2005) have identified four layers that impact on the privacy policy to be adopted:

- ▶ the regulatory regime, which varies substantially between countries;
- ▶ the type of location-based service required;
- ▶ the type of data to be disclosed;
- ▶ the personal attitude towards privacy, which may range from careless to paranoia.

The authors provide a brief overview of various regulatory regimes, classifying them into four cases:

- ▶ states with little or no privacy protection by law;
- ▶ states with some protection, like the USA;
- ▶ states with strong protection, like Canada;
- ▶ states with strong protection and addressing location awareness specifically, like the EU and Japan.



Tracking a mobile phone.

The authors mention that a reason for the proliferation of ubiquitous computing services in Japan is the early definition of privacy regulations for that sector, which led to a high level of consumer confidence. Price et al. (2005) also provide a classification of the ways that the user's location or identity might be concealed, from making it anonymous to providing false information.

Taking all these various factors and issues into account, the challenge is to find the appropriate balance between flexible access to location-dependent information while not giving away sensitive, private information that could be misused. Although Price and colleagues propose a possible model, privacy in ubiquitous and mobile computing is still very much an area of research.

4.4 Developing countries

The highest social impact of mobile technologies is, however, to be expected in developing countries, which may lack comprehensive communication and electricity networks, and where a large part of the population may have low literacy levels. The usage of desktop PCs connected to a fixed network is therefore restricted. Smartphones are a much better alternative and their use has been increasing drastically in many developing countries. For example, in India, mobile subscriptions (not necessarily with internet access) increased by 73% from 2005 to 2006, while landline subscriptions grew by 9% and internet subscriptions by 25% in the same period (Donner, 2006).

Exercise 7

Can you think of some reasons why smartphones are better suited than desktop machines to developing countries?

Discussion.....

Smartphones are battery operated, and battery chargers may be solar powered, making them independent of any electric power network. Smartphones have wireless network access, and it is easier and faster to install a network of transceivers than a network of cables, especially in large countries like India and Mozambique. Smartphones are simpler to use and maintain than PCs; for example, because no complex software installation and configuration is required. Smartphones are cheaper than PCs.

The use of smartphones, together with the provision of financial and business development services targeted to microenterprises (i.e. very small businesses with one or few employees), can have a big impact in the economy, traditions and social fabric of communities.

For example, custom-built software is helping women in rural areas of India to use their mobile phones to keep their accounts and then to text the results to the microfinance firms that provide their loans (Hickey, 2007). The loans and the technology are empowering women in rural India and it will be interesting to watch the social changes this may lead to.

Elsewhere in India, the news provider Reuters piloted a scheme to allow farmers to get weather forecasts and market price information on their mobile phones, in order to improve their crops and sales (O'Connor, 2006). Only time will tell the impact of such technologies on the ancient rituals used to predict the weather.

But mobile technologies do not just affect rural communities. Even in urban centres they may play a role in developing microenterprises. Donner (2006) provides an interesting study of the diversity of needs and attitudes of microenterprises towards information technologies in one of the world's 'IT centres', Bangalore. Donner notices that while mobile phones without internet access may be enough for many microenterprises as a way to reinforce the personal nature of the business–client relationship, others use the internet as a way of obtaining information that may give a competitive edge, as, for example, in the case of a barber who goes to an internet café in order to look for new hair styles. Donner also notes some factors that may facilitate the incremental adoption of IT by such microenterprises:

- ▶ the same device is used for leisure and business purposes;
- ▶ the increased availability of user-generated content in local languages;
- ▶ the convergence of multiple features (calendar, camera, music player etc.) into mobile devices at progressively lower cost.

Activity 12.4

Economic and social impact.

5

Distributed computing: the fallacies and the future

We conclude this unit and the course with a reminder of some of the topics and technologies we have covered throughout the units. We will do that through a brief discussion of a list of issues that distributed system designers and administrators have to take into account, followed by some speculation on what the future might hold.

5.1 The fallacies of distributed computing

During the 1990s, a few senior Sun Microsystems employees concisely summarised eight assumptions that designers of distributed systems repeatedly make and which can lead to all sorts of problems. That list of assumptions is known as ‘the fallacies of distributed computing’.

- 1 The network is reliable.
- 2 Latency is zero.
- 3 Bandwidth is infinite.
- 4 The network is secure.
- 5 Topology doesn't change.
- 6 There is one administrator.
- 7 Transport cost is zero.
- 8 The network is homogeneous.

Calling these assumptions fallacies may be too strong: most software developers and network managers will probably be aware of these issues, given the widespread use of and the many books and courses about distributed systems. Nevertheless, for simplicity of exposition, we will refer to the above list as fallacies.

It is appropriate to discuss these fallacies in this unit, not only because it allows us to weave together some of the topics and technologies covered throughout the course, but also because mobile systems make some of those fallacies more obvious. For example, the fourth assumption is challenged by the ease of eavesdropping in wireless communication.

The network is reliable

Due to both hardware and software failure, and a multitude of other exogenous reasons (e.g. a power failure), we cannot assume that messages sent to nodes in the network will reach their destination. The unreliability of networks has consequences at all levels, from hardware management to application development.

For example, those setting up a wired network and deciding which topology to adopt, how many routers to buy etc. will have to consider how possible failures affect the network: is there a single point of failure that can bring the whole network down?

As another example, application developers must assess how resilient to failure is the chosen software architecture (*Unit 5*, Section 2). They also must deal with the possibility of interrupted communication and not simply assume that a reply will be received for each query. While timeouts (*Unit 5*, Subsection 5.2) are a low-level mechanism to help *detect* broken communication, it is up to the higher-level application logic to *deal* with unreliability. For example, what should the client-side and server-side

applications do if network connection fails while a user is confirming an order on The Music Store website? How can the user know whether the network connection failed before or after the request was sent and hence avoid confirming again the same order, which could lead to double billing?

SAQ 4

In which way do mobile systems reinforce this fallacy?

ANSWER.....

Mobile systems often use wireless communication, which is less reliable than wired communication. For example, a simple movement of a device may interrupt its line-of-sight infra-red communication with another device. Also, radio signals can be subject to interference and be simply cut off, e.g. when going underground.

Latency is zero

Whereas bandwidth has increased substantially over the years, **latency** – the time taken by a communication between two hosts – has not decreased in the same way, and in fact is limited by the speed of light (about 300 km per millisecond). Transferring data between hosts that are 3000 km apart will therefore always take at least 10 milliseconds, no matter how much bandwidth you have, how fast the hosts' CPUs are and how efficient the network protocol is.

One of the dangers of location transparency – as provided by RMI (*Unit 7*) and CORBA (*Unit 11*), for example – is that developers might not be aware of the cost of remote calls, because they 'look' like local calls. Even if each remote call takes just a few milliseconds more (compared to a local call), the latency adds up over many calls, especially if each call transfers just a small amount of data.

We have already alluded to the problem of getting the data granularity right in *Unit 7*, Subsections 5.2 and 5.6.

The current trend towards highly interactive websites just makes this fallacy more important: developing such sites without keeping latency in mind will make them unresponsive and customers will simply go somewhere else. We have seen in *Units 8* and *9* various technologies to make the interaction between client and server more efficient.

Bandwidth is infinite

Bandwidth has risen dramatically in the past years: for example, multi-megabit per second broadband has widely replaced modem internet access in the kilobyte-per-second range. This fallacy may therefore seem to be valid in the 1990s, but no longer today.

More bandwidth makes new applications possible, like multimedia streaming, or different ways of distributing the computation load, like the move from thin to thick clients (*Unit 9*). This increase in bandwidth demands can be somewhat alleviated by suitable software architectures that distribute the load over the network, like P2P (*Unit 5*, Subsection 2.4). The technique of increasing the data granularity can address latency issues, but it increases bandwidth usage.

SAQ 5

How does this fallacy relate to mobile systems?

ANSWER.....

Again, the relationship is due to wireless communication, which has lower bandwidth than wired connections. Moreover, bandwidth is shared among devices in the same cell (Subsection 2.1). As such, the actual bandwidth for each user might be just a small fraction of the overall available bandwidth.

The network is secure

Although awareness about security has risen in the past years, it is still an area of concern, and we have dedicated a whole unit (*Unit 10*) to this topic. As we said before, wireless communication has additional vulnerability, because the radio signals can be received by anyone.

Topology doesn't change

Mobile systems, and in particular ad-hoc networks, explicitly assume that the network topology *does* change, and therefore led (and are still leading) to various innovations, e.g. in terms of efficient routing algorithms. But even the topology of wired networks often changes, e.g. as network administrators introduce new servers and routers in order to cope with growing traffic demands.

Distributed applications should therefore avoid 'hard-wiring' any topological assumptions into the code. Instead they should for example use directory services like LDAP (*Unit 7*) and UDDI (*Unit 11*) or brokers like CORBA (*Unit 11*) to locate the necessary resources, or else use location-transparent schemes like DNS names instead of IP addresses. Naming issues were discussed in *Unit 7*, Section 3.

There is one administrator

Small personal and local area networks (e.g. at home) may have one single administrator, but most networks have various administrators, each one typically responsible for a part of the network, like the database or the web servers. A distributed application is therefore subject to the control of various administrators, who set the disk quotas, the read-write privileges, the available ports, etc. If possible, developers of distributed applications should work together with the administrators of the network where they will be deployed.

Developers may need to provide extra tools that allow administrators to monitor the execution of the application (e.g. for load-balancing purposes) and to diagnose and fix any problems that may occur. Developers also have to think how updates to the distributed application will be deployed. For our music shop scenario, the database schema on the database server has to be updated in a consistent way with the entity classes in the application server, but the two servers are at different locations (one in the shop and the other in the warehouse, see *Unit 1*, Section 7) and hence likely to be under the responsibility of different administrators.

Transport cost is zero

There are two ways in which this fallacy can be interpreted (Rotem-Gal-Oz, 2006). The first way is that going from the application level to the transport level does not involve any costs. This is a fallacy because data has to be marshalled and go through a communication protocol, before the actual transmission over the network takes place (*Unit 5*, Subsection 3.2). This interpretation is basically reinforcing the

'latency is zero' fallacy: the processing done by the various network stacks does take time and requires resources.

The second interpretation is of an economic nature. Setting up and running a network has various costs: servers, routers, security software, etc. have to be bought, administrators have to be hired, bandwidth has to be leased from internet service providers, and so on. Several companies and organisations, like Wikipedia, Google and Flickr, provide services for free, but someone has to pay for the substantial infrastructure and maintenance effort behind those websites. Business models for the internet economy have to be developed. Commonly adopted solutions are donations (Wikipedia), advertisements (Google), or imposing a limit on time, storage quota, etc. for the free service and requiring payment to withdraw such limitations (Flickr).

The network is homogeneous

The last fallacy may be the one that developers and administrators are most aware of: networks connect a bewildering array of different platforms, from large servers to personal desktop machines, all running various operating systems. The variety increases substantially if we consider all kinds of small or entertainment device that are becoming interconnected, from mobile phones to game consoles and TV desktop boxes.

The trick to tackle heterogeneity has been to develop standard communication protocols and notations, of which the Internet Protocol and SOAP are two examples among the many that we have seen in *Units 5 to 11*. The various services and middleware mentioned above to deal with a changing network topology are also a means of providing a homogeneous access to a heterogeneous network.

5.2 The future of distributed computing

Compared to other science and engineering subjects, computing is relatively recent, with the earliest electronic computers being built in the 1940s. Yet, the pace of innovation has been staggering, as the brief historical account in *Unit 1* has shown.

If we just concentrate on distributed computing, we can see that the tendency is to have a substantial increase in the number of CPUs per person. In the early days, a single huge computer in a special room, operated by a large team of dedicated staff, could cater for the calculation needs of various organisations, who could not afford to buy and maintain such a computer for themselves.

In the 1980s, the computer became personal: a person–CPU ratio of 1.

Nowadays, not only entertainment devices that are clearly computational in nature, like MP3 players and game consoles, have CPUs, operating systems and megabytes of application software within them, but also many complex electronic devices, like washing machines and TVs, are controlled by software. Even more complex 'devices', like cars and airplanes, have whole networks within them!

Together, entertainment gadgets and embedded systems have helped to substantially increase the number of CPUs per person, not to mention the use of mobile phones and dual-core laptops. Figure 5 shows schematically the overall trend over time.

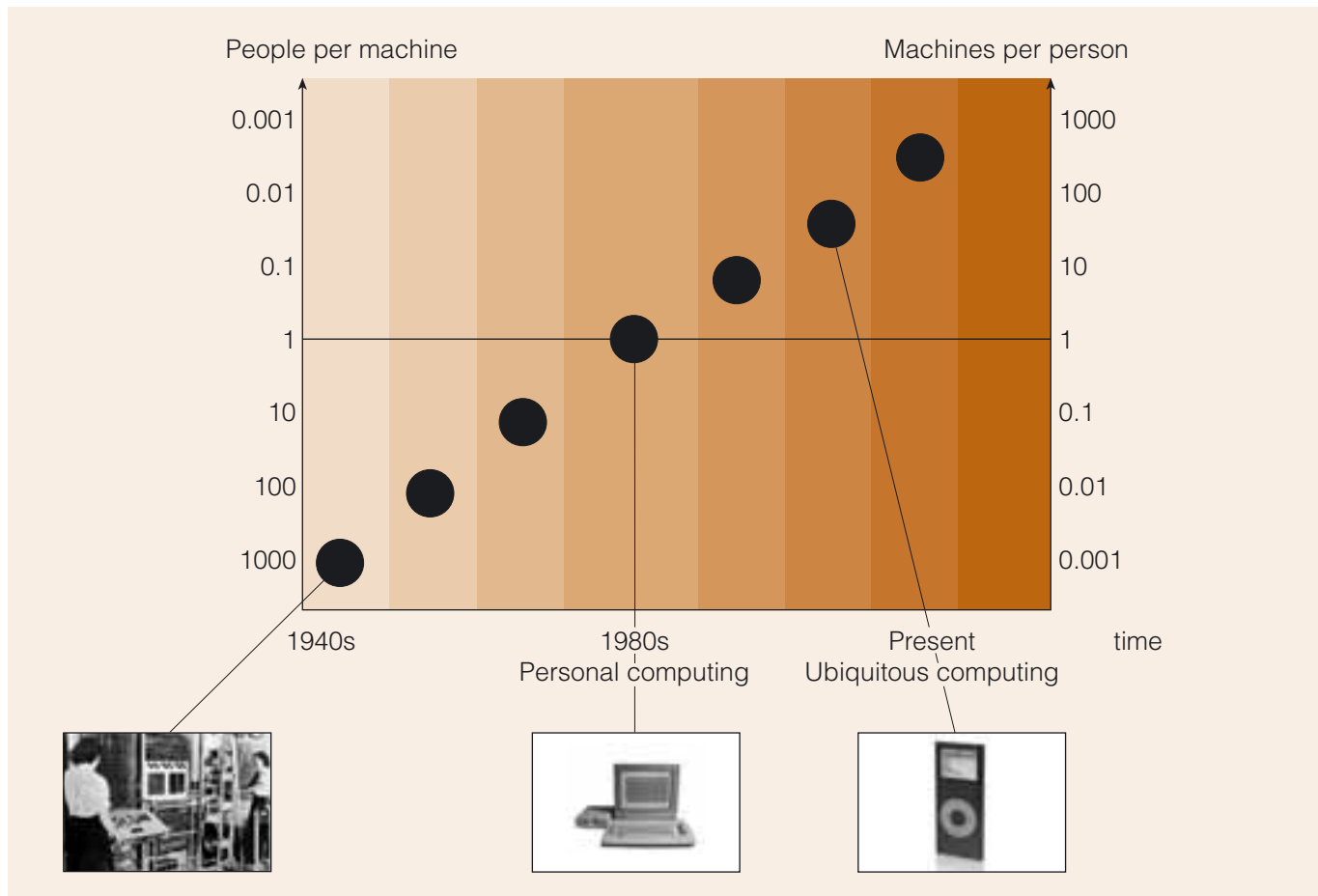


Figure 5 The evolution of the order of magnitude of the ratio of CPUs to people

But where will all this miniaturisation and ubiquity of computing devices lead to? In March 2001, at the beginning of a new millennium, the Association for Computing Machinery organised a conference called 'Beyond cyberspace ... a journey of many directions' and complemented it with approximately 50 essays about the next 1000 years, published in that month's issue of the *Communications of the ACM*. The essays make for thought-provoking reading and cover a wide range of areas, from computer graphics to cyborgs, touching on economic and social issues, hardware and software, and education. We will mention only some of the essays more closely related to distributed systems.

Vetter (2001) concentrates on one single emerging application area: wireless web-based instruction. This will enable students to obtain resources and support for their studies wherever they are. But education will not be only just-in-time, it will be also just-for-you, personalised to the individual student's needs. The small size of devices, in particular of screens, will act as a powerful pressure for applications to filter irrelevant content. Interactivity will be enhanced and voice input, instead of typing text, will be the norm. All this will require high bandwidth and communication reliability, display technologies that allow good readability outdoors, etc.

Waldo (2001) discusses the advantages and disadvantages of the network becoming invisible and ubiquitous. He gives two examples of possible uses. One of them is an extension of the home-appliance engineer carrying around a laptop (Subsection 4.1). In the future, home appliances might be connected directly to the manufacturer's network, thereby allowing the appliance to be monitored and faults to be quickly

diagnosed or even prevented. The second example stems from health care. Sensor networks might monitor our health and advise us to consult a doctor when needed. Waldo also mentions that such networks raise difficult privacy maintenance problems.

Miner (2001) speculates that although the smaller the computational user devices are, the harder they are to use – after all, how small can screens and buttons become? – digital jewellery might be a practical way to ‘blend’ various small devices (earphones, microphone, etc.) with our bodies. Such small devices would mainly address user interaction and communicate wirelessly with each other and with external nodes. For example, computational power and storage would be outsourced to other hosts on the network. Digital jewellery also allows personalisation, a topic that was present in Vetter’s article. The author also notes that, further in the future, advances in nanotechnologies, quantum computers and genetic engineering might make it possible for devices to become so small that they can be injected into or ingested by our bodies. This of course may open the door for software viruses to have biological effects.

Cooper (2001) also wrote a quite futuristic article. He starts by noting that ‘wireless bandwidth has effectively doubled every 2.5 years since Guglielmo Marconi received his first wireless telegraphy patent 105 years ago’. He expects the trend to continue and to allow for an unprecedented amount of information to be streamed into our homes. In particular it will be possible to obtain real-time video and audio with such quality, due to the high bandwidth, that it is indistinguishable from seeing and hearing events ‘live’. The need to get out of the comfort and safety of our homes in order to experience the world and meet other people will diminish greatly. Moreover, due to continuing miniaturisation, keyboards, speakers and large displays will disappear, and will eventually be replaced by implants in our eyes and ears, to be powered by the heat of our bodies, making batteries dispensable.

All this may sound too much like the world depicted in *The Matrix*, a film released two years before the *Communications of the ACM* issue. In the film, the only reality that humans experience is virtual and their only purpose is to be energy providers to machines. Contrary to this bleak view, Miner and Cooper are confident that the technological advances will overall lead to a better society. As Miner puts it: ‘In all, the systems we create allow us to focus instead on those things we enjoy doing – the more creative and social acts that define us as human.’

Activity 12.5

The fallacies and the future of distributed computing.

6

Summary

In this unit we have provided a brief overview of some of the technological, economic and social issues related to mobile systems, an area of rapid development.

There are two kinds of mobility: physical mobility is the change of physical location of hardware components, while logical mobility is the movement of software components among hosts. Mobile systems may involve just one or both kinds of mobility, but each one entails different technological challenges. For example, physical mobility requires efficient and light batteries and cost-effective, non-volatile memories with adequate capacity. It also led to the development of various wireless communication protocols, like WiFi, Bluetooth and ZigBee. On the other hand, logical mobility requires support for at least weak mobility from the underlying platform or middleware, so that programmers can easily develop mobile code applications. We have seen three paradigms for such applications: code on demand, remote evaluation and mobile agents.

Mobile systems can lead to increased productivity and new business opportunities, particularly in developing countries where a comprehensive landline communication infrastructure is missing. Such economic changes have, however, also a social impact, be it a change in work–life balance or in existing traditions and social structures. Mobile systems are also more prone to security attacks and to privacy intrusion than stationary desktop systems within organisational networks. Individual and government attitudes towards such issues vary widely, and there is not yet a consensus on how to address them.

The fallacies of distributed computing highlight some of the themes that are paramount to distributed systems and that have recurred throughout the course: efficiency, reliability, security and heterogeneity. Some of the fallacies are particularly relevant to mobile systems. The fallacies, and the course, also touch on human and economic issues that cannot be addressed by software alone, yet impact on the design of systems. Such issues become even more important in some of the views about the possible future of distributed systems.

Concurrency and distribution have been studied for decades by computer scientists, yet these topics are more relevant than ever, as technological advances make it possible to manufacture in a cost-efficient way millions of small networked devices that disappear into the fabric of modern life. This course has aimed to give you the knowledge and the intellectual and practical skills necessary to keep abreast of this exciting area of rapid development in computing, and we hope it has also inspired you to do so.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ explain the difference between physical and logical mobility and illustrate it with their differing technological challenges;
- ▶ explain and illustrate the essential properties of mobile computing;
- ▶ know some of the existing communication technologies and protocols developed for mobile systems and appreciate their different purposes;
- ▶ explain what location awareness is, give examples of how it can be achieved, and discuss its benefits and pitfalls;
- ▶ explain the difference between strong and weak mobility and why Java does not support the former;
- ▶ explain the three mobile code paradigms, give examples of their current use, and argue for their relative benefits and problems;
- ▶ discuss some of the economic and social impact of mobile systems, including privacy and security issues;
- ▶ explain the eight fallacies of distributed computing and why they are still relevant, in particular for mobile systems, more than a decade after being formulated.

Further terminology concerning mobile systems can be found in the Glossary of *Unit 1*.

Glossary

ad-hoc network A network with dynamic routing and without dedicated nodes like routers and hubs, thereby allowing nodes to easily join and leave the network.

application portability The ability to adapt an application to a different platform. The easier the adaptation is, the more portable the application is said to be.

cell (of a transceiver) The area that a transceiver covers.

code on demand On a request from the client, code is transferred from the server and executed on the client, using the client's resources.

context awareness The ability for hardware to detect the current environment and for software to adapt to it.

geotagged information Data that has been associated to a physical location, e.g. the address of a restaurant is geotagged with the latitude and longitude of its location.

Global Positioning System (GPS) A satellite system that sends continuous radio signals, thereby enabling a receiver on the Earth to compute its latitude and longitude.

latency In general, the delay between the time an activity is initiated and the time its effects are noticed. In the concrete case of networks, it is the time taken for messages to be transmitted from one host to another.

logical mobility The change of host of a software component.

mobile agent An execution unit that moves, together with some code and resources, from one site to another, where it resumes execution.

mobile code Code that is transferred from one execution unit to another, where it is executed.

mobile computing The use of a portable computer with a wireless network connection.

Mobile IP A standard internet protocol for communicating with mobile nodes.

personal area network A short-range network connecting various devices, typically for personal use, with each other; for example, a network connecting home entertainment devices, a personal computer and a mobile phone.

physical mobility The change of physical location of a hardware component.

physical portability The ability to easily carry and move a computing device from one physical location to another.

portability In the context of mobile systems, this means physical portability. In the context of software development, it means application portability.

radio-frequency identification (RFID) A radio-based technology to communicate with small devices that can store and change small amounts of data.

remote evaluation The transfer of code from the client to the server, where it is executed using local resources, after which a reply is sent to the client.

strong mobility The ability, provided by the platform or middleware, to suspend an execution unit, capture its state (including the code it is executing and the data it is manipulating) and transfer all this to a new site, where execution is resumed.

transceiver A transmitter and receiver of radio wave signals.

weak mobility The ability, provided by the platform or middleware, to transfer code from one site to another.

References

- BBC (2007) 'Text messages warn of clip joints' [online],
<http://news.bbc.co.uk/1/hi/england/london/6971871.stm> (Accessed 28 January 2008).
- Carzaniga, A., Picco, G.-P. and Vigna, G. (2007) 'Is code still moving around? Looking back at a decade of code mobility' in *Proceedings of the 29th International Conference on Software Engineering*, IEEE.
- Cooper, M. (2001) 'Bandwidth and the creation of awareness', *Communications of the ACM*, vol. 44, no. 3, pp. 55–7.
- Donner, J. (2006) *Internet Use (and Non-Use) among Urban Microenterprises in the Developing World: An Update from India* [online], Seventh Association of Internet Researchers conference,
http://www.jonathandonner.com/jdonner_AoIR_microenterprise.pdf (Accessed 28 January 2008).
- Forman, G. and Zahorjan, J. (1994) 'The challenges of mobile computing', *IEEE Computer*, April, pp. 38–47.
- Ghosh, D. (2000) 'Mobile IP' *ACM Crossroads* [online], vol. 7, no. 2,
<http://www.acm.org/crossroads/xrds7-2/mobileip.html> (Accessed 28 January 2008).
- Hickey, H. (2007) 'High tech for global justice: UW computer scientist designs custom cell phones' [online], <http://uwnews.org/uweek/uweekarticle.asp?articleID=31032> (Accessed 28 January 2008).
- Huebscher, M., Pryce, N., Dulay, N. and Thompson, P. (2006) 'Issues in developing ubicomp applications on Symbian phones' in *Proceedings of the International Workshop on System Support for Future Mobile Computing Applications*, IEEE.
- Jaokar, A. and Fish, T. (2006) *Mobile Web 2.0*, Futuretext.
- Miner, C. (2001) 'Pushing functionality into even smaller devices', *Communications of the ACM*, vol. 44, no. 3, pp. 72–3.
- O'Connor, A. (2006) 'India's farmers switch faith to mobile phones' [online], *Times Online*, 24 July,
<http://business.timesonline.co.uk/tol/business/markets/india/article691684.ece> (Accessed 28 January 2008).
- Price, B.A., Adam, K. and Nuseibeh, B. (2005) 'Keeping ubiquitous computing to yourself: a practical model for user control of privacy', *International Journal of Human-Computer Studies*, vol. 63, nos 1–2, pp. 228–53.
- Rotem-Gal-Oz, A. (2006) *Fallacies of Distributed Computing Explained* [online],
<http://www.rgoarchitects.com/Files/fallacies.pdf> (Accessed 28 January 2008).
- Sandholm, T. and Huai, Q. (2000) 'Nomad: mobile agent system for an internet-based auction house', *IEEE Internet Computing*, March/April, pp. 80–6.
- Vetter, R. (2001) 'The wireless web', *Communications of the ACM*, vol. 44, no. 3, pp. 60–1.
- Waldo, J. (2001) 'When the network is everything', *Communications of the ACM*, vol. 44, no. 3, pp. 68–9.
- Want, R. and Schilit, B. (2001) 'Expanding the horizons of location-aware computing', *IEEE Computer*, vol. 34, no. 8, pp. 31–4. [Guest editors' introduction to the special section on location-aware computing.]
- Weiser, M. (1991) 'The Computer for the 21st Century', *Scientific American*, vol. 265, no. 3, pp. 94–104.

Acknowledgements

Grateful acknowledgement is made to the following sources:

Figures 1–4: Cartoons based upon those used by A. Carzaniga, G.-P. Picco and G. Vigna in their presentation 'Is code still moving around? Looking back at a decade of code mobility', at the IEEE International Conference on Software Engineering 2007.

Figure 5: Image of Colossus: Taken from www.wikipedia.org. Original source Public Record Office, London.

Image of Commodore 64c: Taken from www.wikipedia.org. Copyright © Bill Bertram

Image of second generation iPod nano: Taken from www.wikipedia.org. Copyright © IE

Every effort has been made to contact copyright holders. If any have been inadvertently overlooked, the publishers will be pleased to make the necessary arrangements at the first opportunity.

Index

A

Active Badge 11

ad-hoc network 8

B

bandwidth 8

C

cell (of a transceiver) 7

chocolate-cake baking model 14–21

client–server paradigm 15

code on demand 18

mobile agent 20

remote evaluation 16

context awareness 13

E

Enhanced 911 12

error control (wireless communication) 8

execution unit 14

F

fallacies of distributed computing 28

administration 30

bandwidth 29

homogeneity 31

latency 29

reliability 28

security 30

topology 30

transport 30

G

geotagging 12

Global Positioning System (GPS) 12

I

internet business models

Flickr 31

Google 31

Wikipedia 31

L

location awareness 10

London congestion charge 11

M

mobile

agent 20

code 14

computing 7

mobile devices

economic impact 24

social impact 24

Mobile IP 10

mobility

logical 5, 14, 34

physical 5, 34

strong 22

weak 22

N

node identity 9

care-of address 10

fixed address 10

foreign agent 10

home agent 10

Nomad (mobile agent system) 22

P

personal area network 8

portability 7

application 7

physical 7

privacy intrusion 25

R

radio-frequency identification (RFID) 11, 25

S

security 8, 10

smartphone 24

Symbian phone 13

T

text messaging 24

transceiver 7

V

Via Verde 11

W

- wireless communication 7
 - infra-red transmission 7, 11, 29
 - microwave transmission 7, 12
 - radio transmission 7, 10, 12, 29
- wireless sensor network 8

- wireless sensor network protocols
 - Bluetooth 9, 25
 - WiFi 8–9
 - WiMAX 8
 - ZigBee 9, 25

